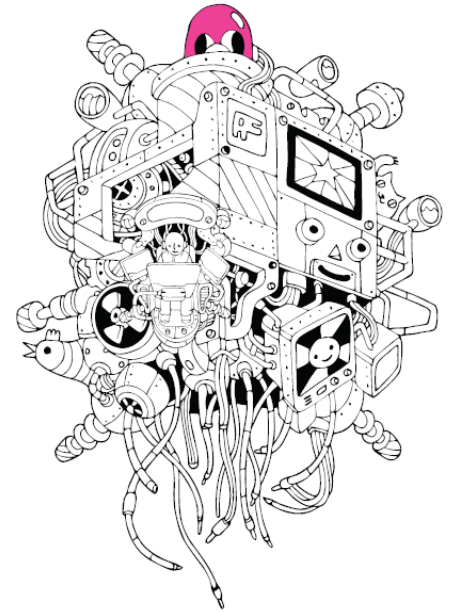


C programming



C Programming:

Chapter 07. Statements that command repeated execution

C programming



Chapter 07-1. While statement

C Programming:

Understanding loops and “while” statements

- What is a loop?

A sentence that is composed to execute one or more sentences more than once.

- Types of loops

while, do~while, for

If the target of repetition is a single sentence, the curly brackets can be omitted.

```
while(num<5)
    printf("Hello world! %d \n", num++);
```

```
while(num<5)
    printf("Hello world! %d \n", num), num++;
```

```
int main(void)
{
    int num=0;
    while(num<5)
    {
        printf("Hello world! %d \n", num);
        num++;
    }
    return 0;
}
```

while loop

Repeated area inside curly brackets

```
Hello world! 0
Hello world! 1
Hello world! 2
Hello world! 3
Hello world! 4
```

Execution results

The object that becomes the purpose of repetition
The variable num is to control the number of repetitions !



Indentation is also used within loops.

Not indented

```
int main(void)
{
int num=0;
while(num<5)
{
printf("Hello world! %d\n", num);
num++;
}
return 0;
}
```

Indented

```
int main(void)
{
    int num=0;
    while(num<5)
    {
        printf("Hello world! %d\n", num);
        num++;
    }
    return 0;
}
```

The difference between indented and unindented is easily noticeable !



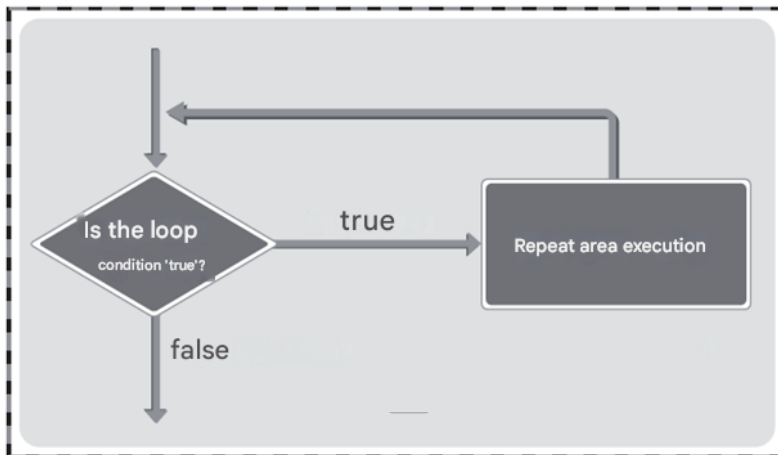
A detailed look at the structure and execution flow of “while”

```
int main(void)
{
    int num=0;
    while(num<3)  // 3회 반복
    {
        printf("Hello world! %d \n", num);
        num++;
    }
    . . . .
}
```



Appendix

What is the process of repetition ?



while statement

Output of multiplication tables

```
int main(void)
{
    int times=0, num=1;
    printf ("which times table?");
    scanf("%d", &times);

    while(num<10)
    {
        printf("%d×%d=%d \n", times, num, times*num);
        num++;
    }
    return 0;
}
```

which times table? 7

7 × 1 = 7
7 × 2 = 14
7 × 3 = 21
7 × 4 = 28
7 × 5 = 35
7 × 6 = 42
7 × 7 = 49
7 × 8 = 56
7 × 9 = 63

Execution results

The multiplication table is a representative example used to understand loops.
Later, in nested loops, we will see an example of printing the entire multiplication table.



Nested while loops

ch07_while.c

while statement exists within a while statement. In the example below, a while statement is nested to print the entire multiplication table. Let's understand the code flow of nested while statements through this example.

```
int main(void)
{
    int cur=2;
    int is=0;

    while(cur<10) // Repeat from 2 to 9 times
    {
        is=1; // To start a new section

        while(is<10) // Express the product of 1 to 9 for each table
        {
            printf("%d×%d=%d \n", cur, is, cur*is);
            is++;
        }
        cur++; // Increment to move to the next level
    }

    return 0;
}
```

outer while statement

inner while statement

C programming



Chapter 07-2. Using the do~while statement repetition of sentences

C Programming:

Basic structure of “do~while”

```
do
{
    printf("Hello world! \n");
    num++;
} while(num<3);
```



Appendix

What is the process of repetition ?

Because the repetition condition is performed at the end of the loop,

The loop will be executed **at least once**. This is the biggest difference from the while statement.



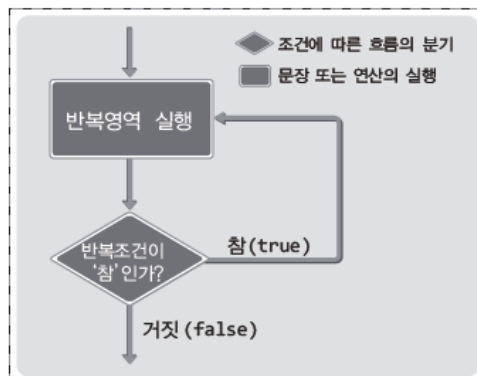
do~while statement

ch07_dowhile.c

```
while(num<10)
{
    printf("%d×%d=%d \n", dan, num, dan*num);
    num++;
}
```

↕ Loops that repeat the same number
of times

```
do
{
    printf("%d×%d=%d \n", dan, num, dan*num);
    num++;
} while(num<10);
```



do~while statement
flowchart

```
int main(void)
{
    int total=0, num=0;
    do
    {
        printf("Enter integer (0 to quit): ");
        scanf("%d", &num);
        total += num;
    }while(num!=0);
    printf("Total: %d \n", total);
    return 0;
}
```

Execution results

```
Enter integer (0 to quit): 1
Enter integer (0 to quit): 2
Enter integer (0 to quit): 3
Enter integer (0 to quit): 4
Enter integer (0 to quit): 5
Enter integer (0 to quit): 0
Total: 15
```

It is natural to construct a loop that must be executed at least once as a do-while loop.

C programming



Chapter 07-3. For loop
repetition of sentences

C Programming:

3 essential elements

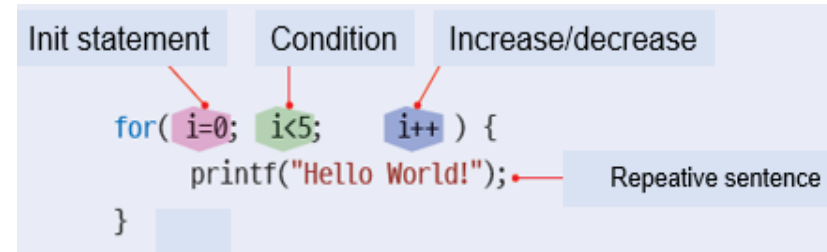
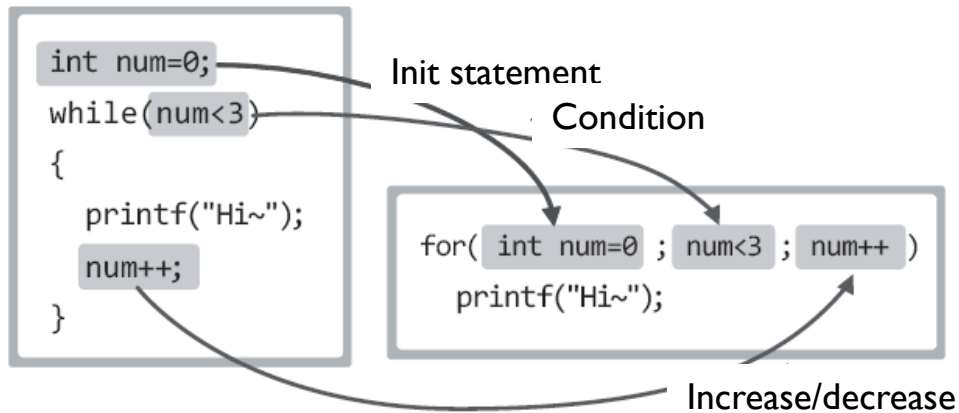
```
int main(void)
{
    int num=0;    // Essential Element 1. Declaration of variables for iteration
    while(num<3)  // Essential Element 2. Checking the loop condition
    {
        printf("Hi~");
        num++;    // Essential Element 3. Operation to make the loop condition 'False'
    }
    . . . .
}
```

1. For a fixed number of repetitions, one variable is required .
2. A conditional check based on that variable is required .
3. An operation is required to make the condition check false .

The **for** loop helps us display these three things on one line.

the while loop above, the three elements required for the loop are spread across multiple lines. Therefore, the number of repetitions cannot be recognized immediately.

Structure and Understanding of “for”



Appendix

```
int main(void)
{
    int num;
    for(num=0; num<3; num++)
        printf("Hi~");
    . . .
}
```

If for statement is one line, the curly brackets can be omitted !



Understanding the flow of “for”

Components of for statement

- ✓ The **initial expression** is executed exactly once before the full iteration begins.
- ✓ The **conditional expression** is executed before the start of each repetition,
and the repetition is determined based on the result !
- ✓ The **operation** is performed at the end of each iteration of the **increment/decrement expression**.

for loop flow

int num = 0: The initialization corresponding to `int num=0` is performed exactly once before the loop starts!

num<3: The condition check corresponding to `num<3` is performed before the start of every loop!

num++: The increment/decrement operation corresponding to `num++` is performed after the loop body has finished executing!

➡ Flow of the first iteration

1 → 2 → 3 → 4 [num=1]

➡ Second iteration flow

2 → 3 → 4 [num=2]

➡ Third iteration flow

2 → 3 → 4 [num=3]

➡ Fourth iteration flow

2 [num=3] Therefore, escape!

```
for( 1int num=0 ; 2num<3 ; 4num++ )  
{ 3  
    printf("Hi~");  
}
```

Various examples based on “for”

```
int main(void)
{
    int total=0;
    int i, num;
    printf("Addition from 0 to num, what is num? ");
    scanf("%d", &num);

    for(i=0; i<num+1; i++)
        total+=i;

    printf("Addition result from 0 to %d: %d \n", num, total);
    return 0;
}
```

Addition from 0 to num, what is num? 10
Addition result from 0 to 10: 55

Execution results

“,” the initial expression, condition expression, and increment/decrement expression can be omitted if unnecessary.
If the condition is omitted, it is recognized as true and an infinite loop is formed.

Execution results

Floating-point input (minus to quit): 3.2323
Floating-point input (minus to quit): 5.1891
Floating-point input (minus to quit): 2.9297
Floating-point input (minus to quit): -1.0
Average: 3.783700

for loop through various examples !

```
int main(void)
{
    double total=0.0;
    double input=0.0;
    int num=0;

    for( ; input>=0.0 ; )
    {
        total+=input;
        printf("Enter a real number (minus to quit): ");
        scanf("%lf", &input);
        num++;
    }

    printf("Average: %f \n", total/(num-1));
    return 0;
}
```

Nested for loops with “for”

```
int main(void)
{
    int cur, is;

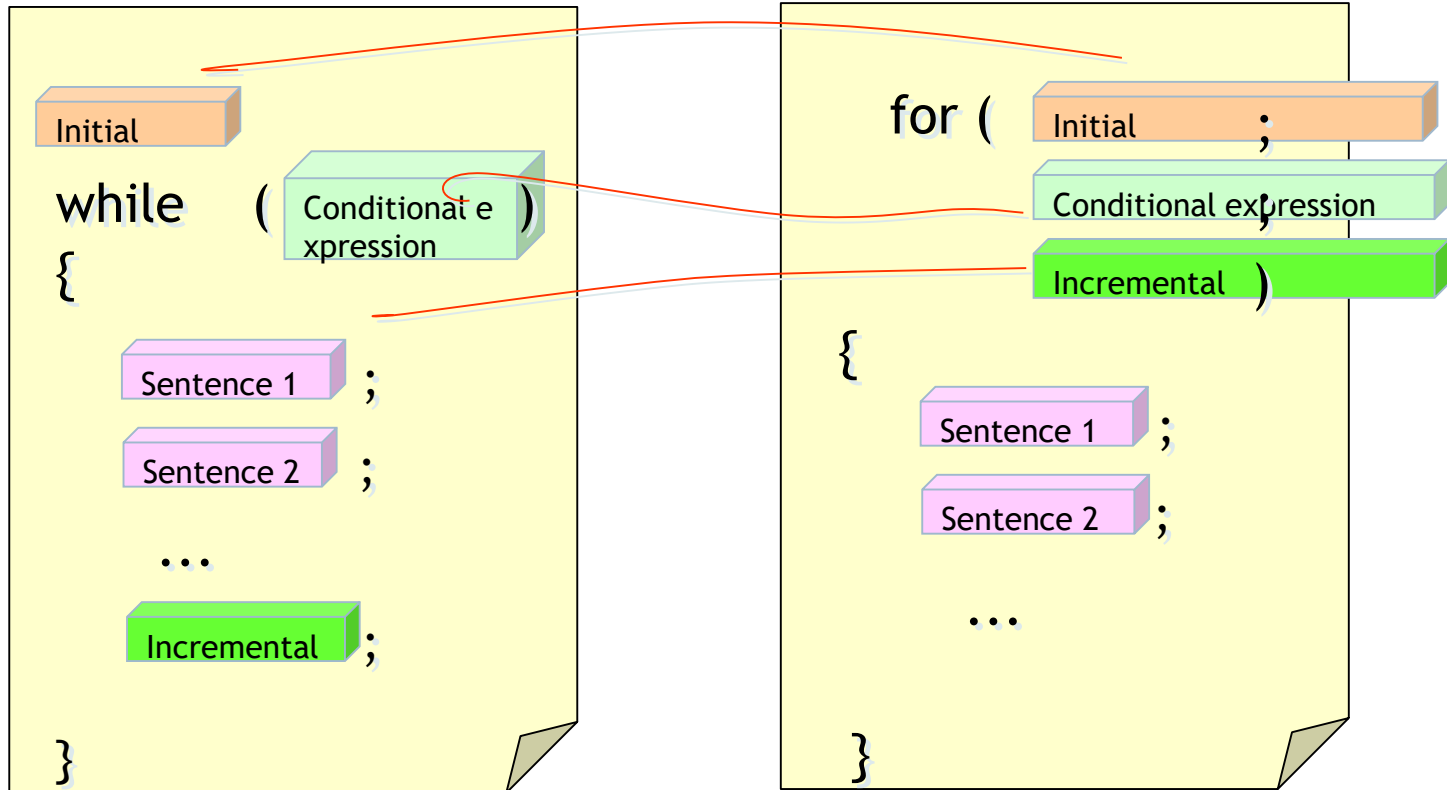
    for(cur=2; cur<10; cur++)
    {
        for(is=1; is<10; is++)
            printf("%d×%d=%d \n", cur, is, cur*is);
        printf("\n");
    }
    return 0;
}
```

Nesting for loops is no different from nesting while and do~while loops.

Let's understand nesting of for loops through the example on the left that prints the entire multiplication table.



Comparison of while, do-while, and for loops



Various forms of increasing and decreasing formulas

```
for ( int i = 10; i > 0; i -- )  
    printf ( "Hello World!\n" );
```

Using subtraction

```
for ( int i = 0; i < 10; i += 2 )  
    printf ( "Hello World!\n" );
```

Increase by 2

```
for ( int i = 1; i < 10; i *= 2 )  
    printf ( "Hello World!\n" );
```

Multiply by 2

```
for ( int i = 0; i < 100; i = ( i * i ) + 2 )  
    printf ( "Hello World!\n" );
```

Any formula is possible



Various forms of increasing and decreasing formulas

```
for ( ; ; )  
    printf ( "Hello World!\n" );
```

Infinite loop

```
for ( ; i<100; i++ )  
    printf( "Hello World!\n" );
```

One part may be missing

```
for ( i = 0, k = 0; i < 100; i++ )  
    printf( "Hello World!\n" );
```

two or more variables

```
for ( printf ( " loop start " ), i = 0; i < 100; i ++ )  
    printf ( "Hello World!\n" );
```

Any formula is possible

```
for ( i = 0; i < 100 && sum < 2000; i++ )  
    printf ( "Hello World!\n" );
```

Any complex expression can be a conditional expression.



Various for loop applications

ch07_for.c

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int x, y;
```

```
    for(y = 0; y < 5; y++)
```

```
    {
```

```
        for(x = 0; x < 10; x++)
```

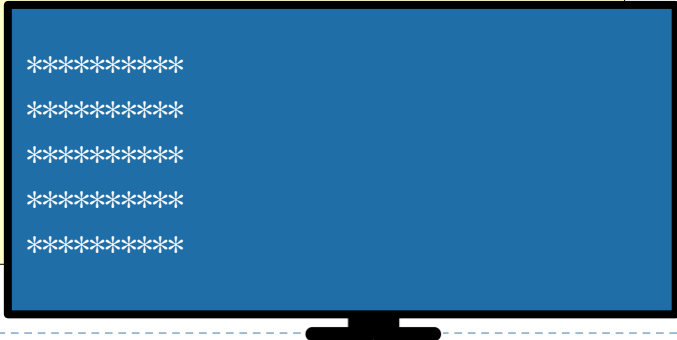
```
            printf("*");
```

```
            printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```



```
*****  
*****  
*****  
*****  
*****
```

Conventional form

```
while(i != 0)
{
    ...
}
```



```
while( i )
{
    ...
}
```

caution



beware of errors

If you use a semicolon(;) at the end of the while condition, only NULL statements are repeated. A statement that only contains a semicolon is called a NULL statement

```
while (i<10) ;
```

It is treated as one sentence and only this is repeated.

```
i++;
```

It doesn't repeat.

```
while(i = 2)  
{  
    ...  
}
```

Since the value of the expression is 2, it is always true and thus causes an infinite loop.

Structure of an infinite loop

Think over....

```
while( 1 )
{
    printf("%d×%d=%d \n", dan, num, dan*num);
    num++;
}
```

While(n < 2) ; {...}

while(n = 2) {...}

The number 1 means ' true ', so the condition of the loop continues to be ' true '.

In this way, when the escape condition of a loop is not met, it is said to form an infinite loop .

These infinite loops are sometimes created accidentally , but they can also be useful in conjunction with break statements.



Infinite loop

- In a conditional control loop, sometimes the program repeats itself infinitely. This is known as an infinite loop. This is a problem because when an infinite loop occurs, the program cannot escape from it.
- But sometimes, intentionally Infinite loops are used because, for example, a traffic light control program must repeat infinitely.

Syntax

infinite loop statement

Syntax

```
while (1) {  
    if (condition)  
        break;    # Stop repetition.  
    if (condition)  
        continue; # Start the next iteration.  
}
```


When infinite loops are useful

- It is especially used in cases where the conditions for exiting the loop are tricky. For example, let's say you need to exit the while loop if the number entered by the user is a multiple of 3 or a negative number.

```
while((x % 3 != 0) && (x >= 0)) {  
    ...  
    ...  
    ...  
}
```

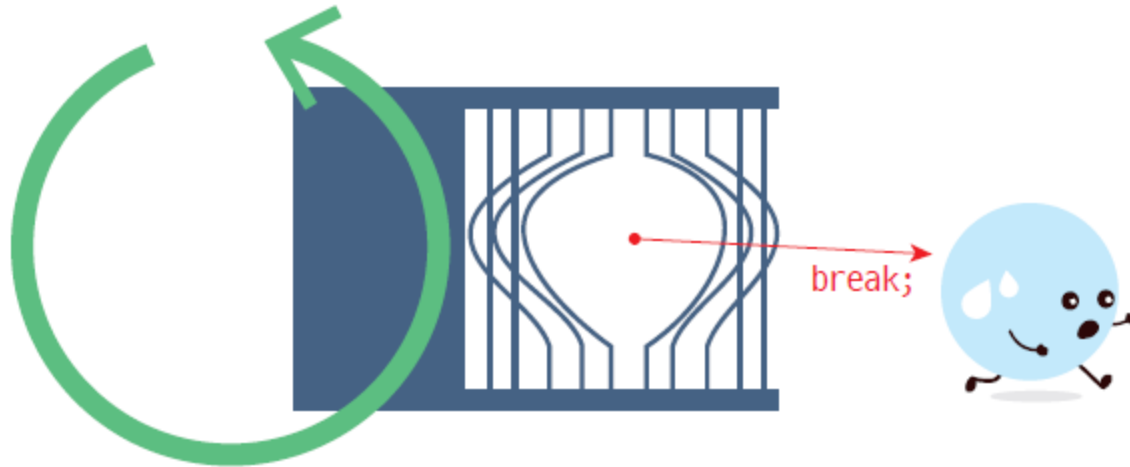


```
while (1) {  
    if (x%3 == 0) break;  
    if (x<0) break;  
    ...  
}
```

You should understand what this code mean

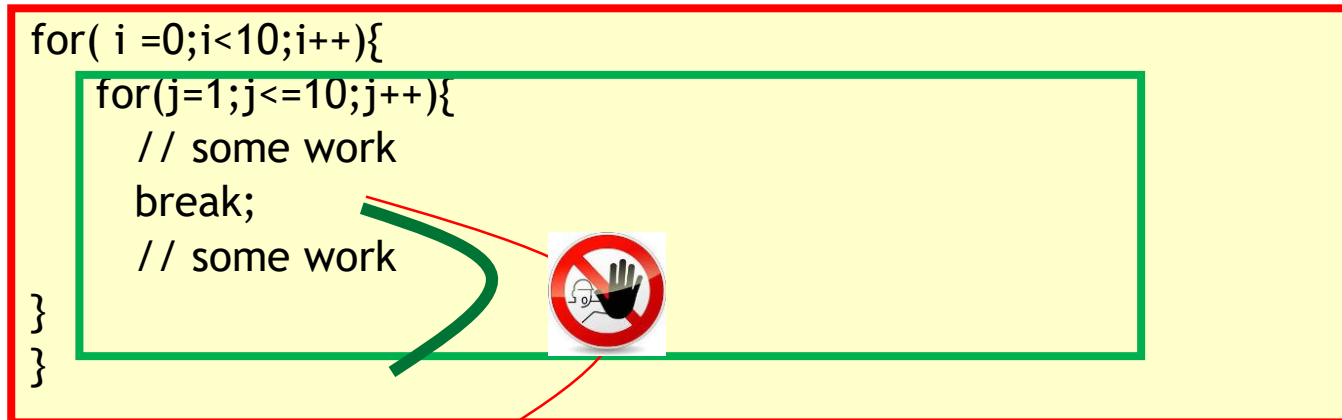
“break” statement

- break statement is used to exit a repeating loop.



The only case you need “goto” statement

- If a problem occurs inside a nested loop , you can use **goto** to quickly **exit the whole loop**.
- **break** If you use it, you can **only break one loop**.



“goto” statement

```
#include <stdio.h>
```

```
int main( void )
```

```
{
```

```
    int x, y;
```

```
    for (y = 1; y < 10000; y++)
```

```
{
```

```
        for (x = 1; x < 50; x++)
```

```
{
```

```
            if ( _kbhit () ) goto OUT;
```

```
            printf ( "*" );
```

```
        }
```

```
        printf ( "\n" );
```

```
    }
```

```
OUT:
```

```
    return 0;
```

```
}
```

```
*****  
*****  
*****
```

“continue” statement

- Print all integers from 0 to 10, excluding those that are multiples of 3.

```
#include <stdio.h>

int main( void )
{
    int i;
    for ( i=0 ; i<10 ; i++ )
    {
        if ( i%3 == 0 )
            continue;
        printf ( "%d ", i );
    }
    return 0;
}
```

1 2 4 5 7 8

“continue” statement

while (conditional expression)

{

sentence;

sentence;

continue

sentence;

}

do

{

sentence;

sentence;

continue

sentence;

} while (condition);

for (initial expression;

conditional expression

;

Incremental

)

{

sentence;

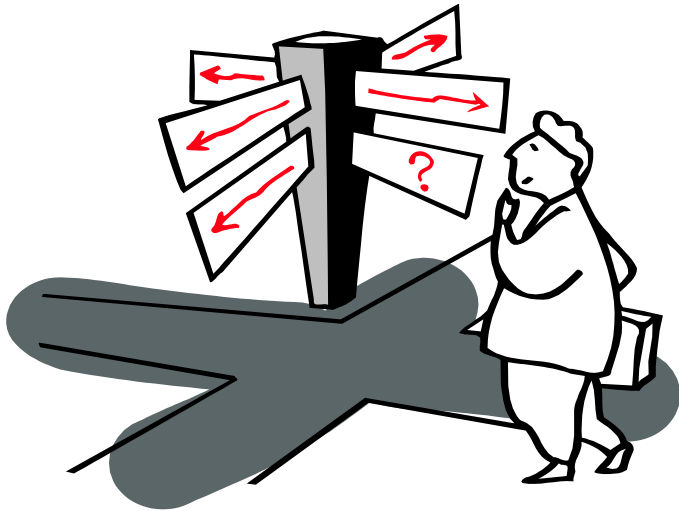
sentence;

continue

sentence;

}

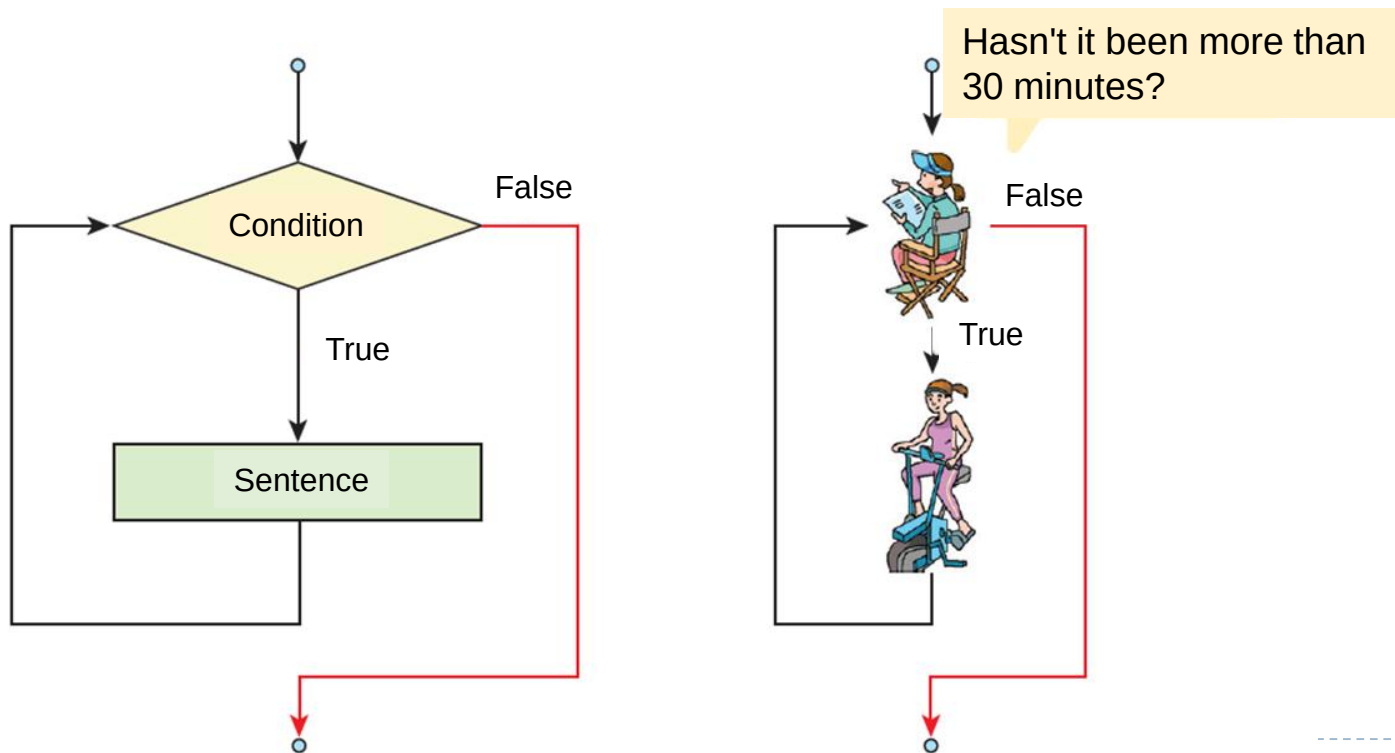




Chapter 07 This That's it . Do you have any questions ?

while statement

Repeats execution of statements while a given condition is satisfied.



“while” statement

Syntax

while statement

yes

```
while( i < 10 ) {  
    printf("Hello World!\n");  
    i++;  
}
```

Conditional statement

If the condition is true, the statement is executed repeatedly.



Example

```
#include <stdio.h>
int main(void)
{
    int i = 0;
    while( i < 5 )
    {
        printf("Hello World! \n");
        i++;
    }

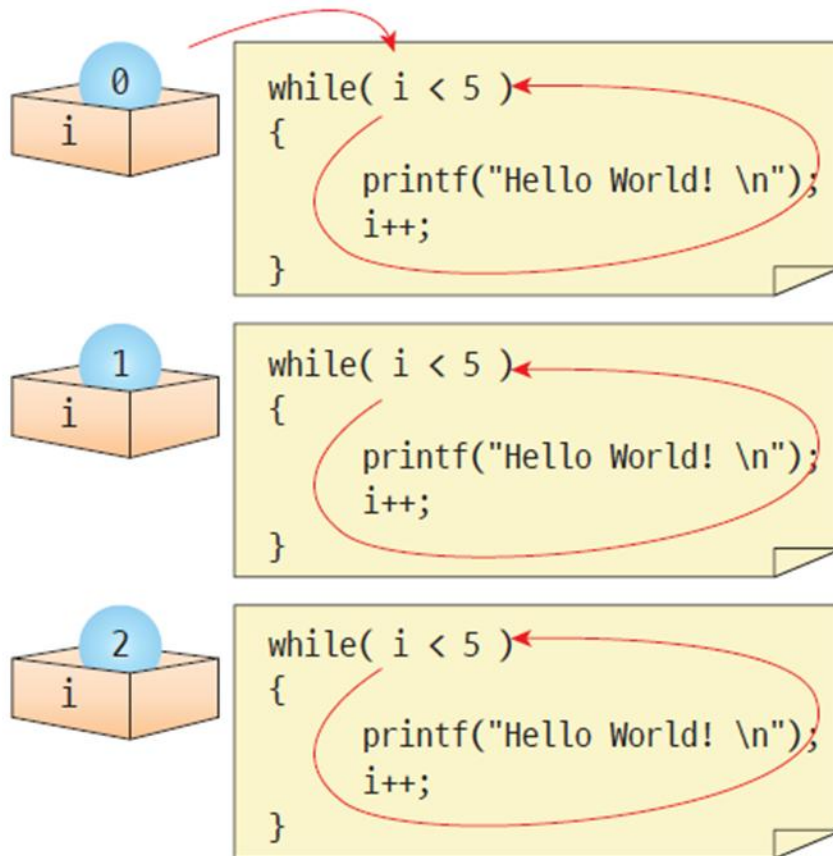
    return 0;
}
```

Repeat condition

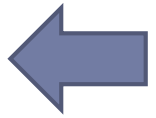
Repeat content

Hello World!
Hello World!
Hello World!
Hello World!
Hello World!

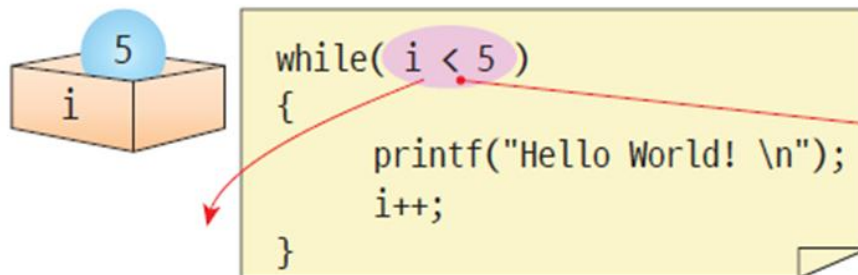
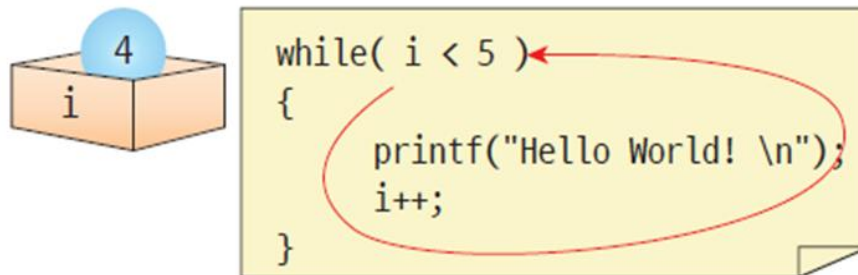
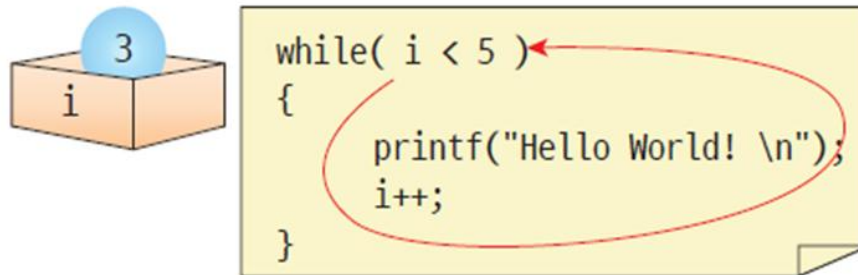
while statement execution process



Number of repeat	Value of i	(i<5)	Repeat or not
#1	0	True	repeat
#2	1	True	repeat
#3	2	True	repeat
#4	3	True	repeat
#5	4	True	repeat
#6	5	False	stop



while statement execution process

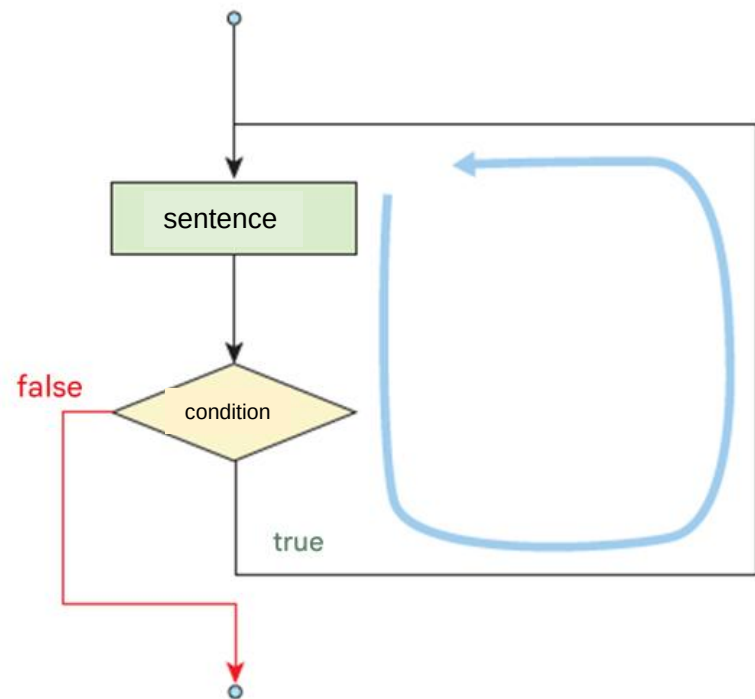
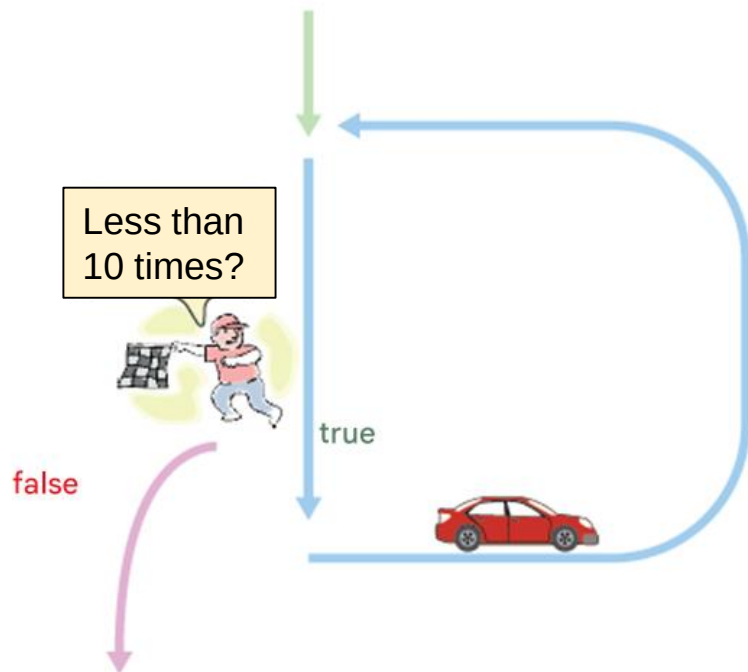


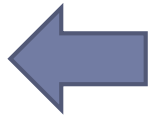
Number of repeat	Value of i	(i<5)	Repeat or not
#1	0	True	repeat
#2	1	True	repeat
#3	2	True	repeat
#4	3	True	repeat
#5	4	True	repeat
#6	5	False	stop

The loop stops when the condition becomes false.

“do-while” statement

◀ Executes the loop statement **at least once**.





“do...while” statement

Syntax

do...while statement

yes

```
do  
{  
    sentence 1;  
    Sentence 2;  
    ...  
} while (condition);
```

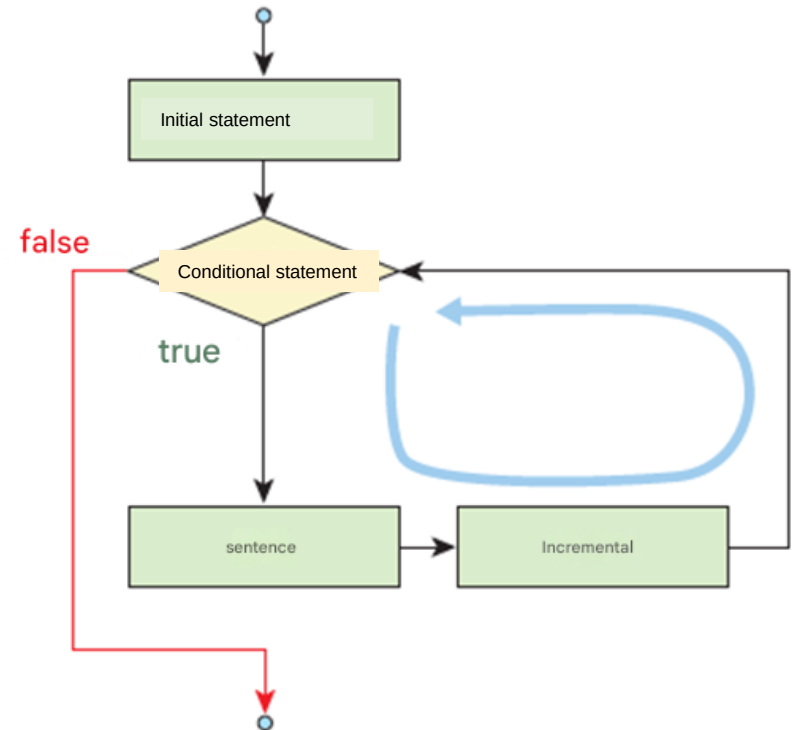
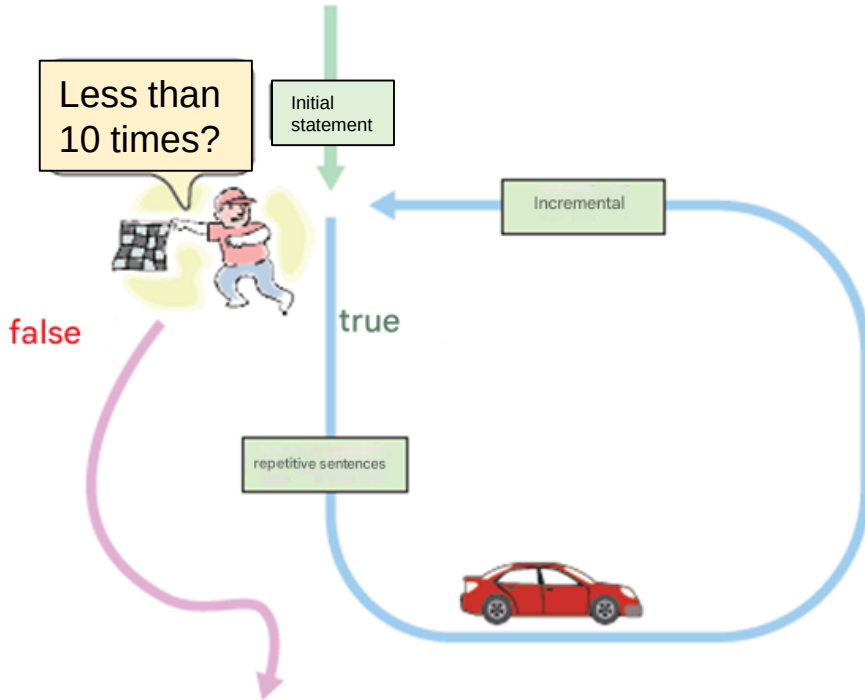
Repeated sentence

If the condition is true, the loop continues.



for loop

◀ A structure that repeats a set number of times



Structure of “for” statement

Syntax

for 문

예

Init statement

Condition

Increase/decrease

```
for( i=0; i<5; i++ ) {  
    printf("Hello World!");  
}
```

Repeative sentence



Initial expression, conditional expression, incremental expression

- Initial

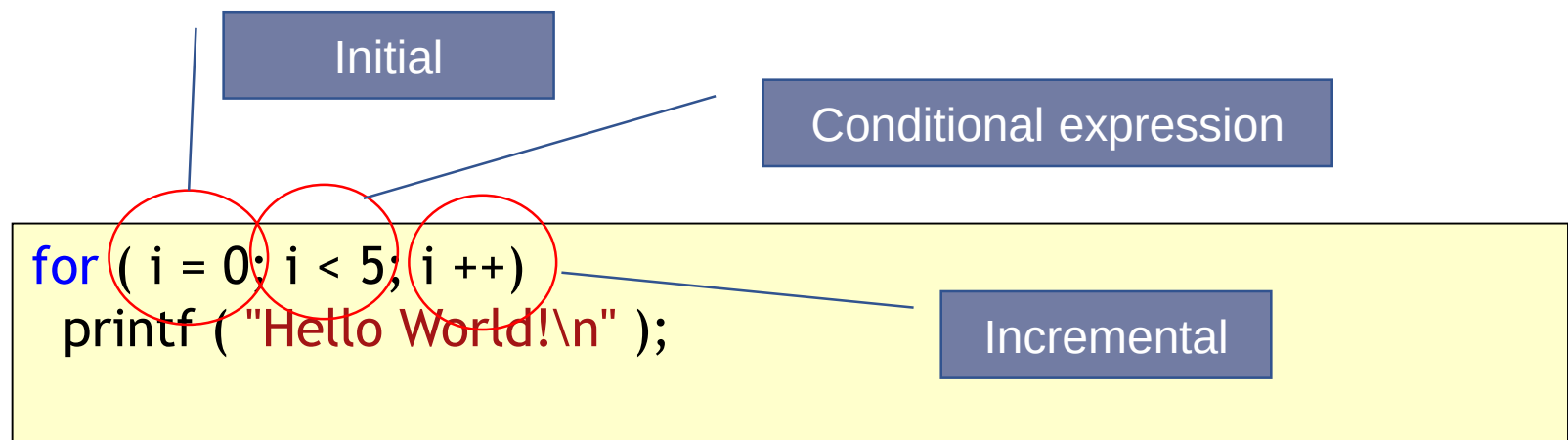
. The initialization expression is executed only once before starting the repeating loop. It is mainly used to initialize variable values.

- Conditional expression

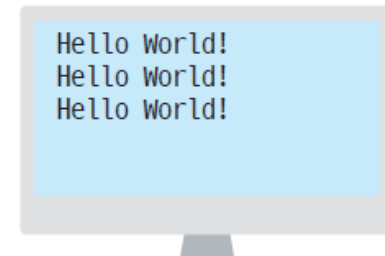
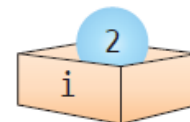
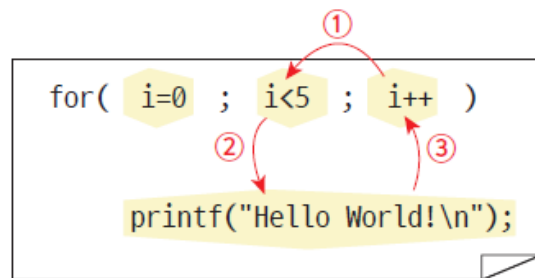
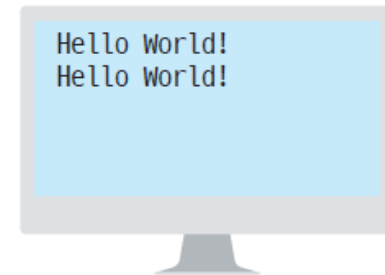
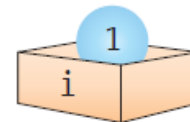
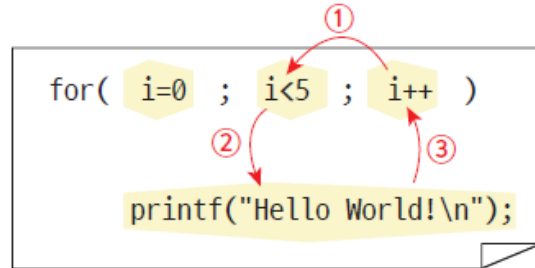
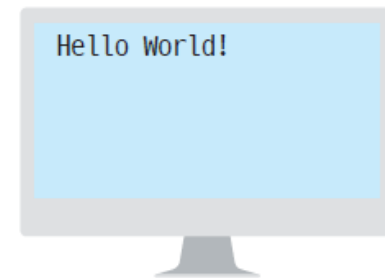
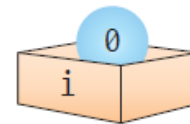
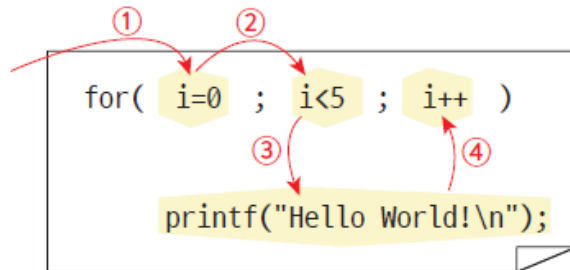
. A formula that checks the condition for repetition. If the value of this expression becomes false, the repetition stops.

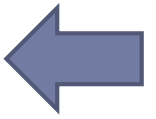
- Incremental

. After one loop execution is complete, the increment expression is executed.



The execution process of “for”





The execution process of “for”

